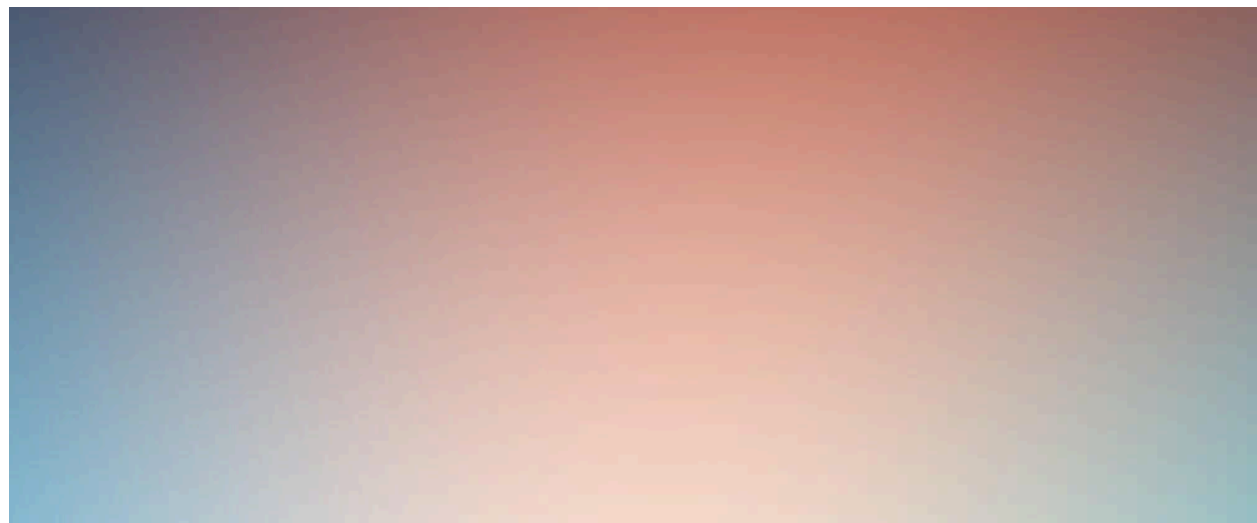




Designing support experience for onprem systems with no reliable internet

🕒 Created	January 3, 2026 11:18 AM
🏷️ Tags	Product Design UX

Background

Qure is a B2B AI healthcare company. Its product, qTrack, is an AI-based chest X-ray screening platform used in TB screening camps across low resource, low-connectivity environments. These camps are run day to day by a range of healthcare workers - radiologists, radiographers, TB nurses, doctors.

Problem

Leadership raised concerns about the high number of support tickets being raised by clients. It was creating a poor customer experience and client trust on the app, and a significant bandwidth of CS and TechOps was going into resolving these issues manually. Every quarter from 45 - 53 % of the tickets were L1 category (basic leave issues which can be resolved by users themselves).

The analysis product and engineering team shared they attributed to client side mistakes rather than product faults. They framed this as a communication and knowledge gap: users did not fully understand how to use the qTrack, so this was a training problem, not something that is going wrong on the product. Whatever wrong is happening because users fully don't understand the product and it is leading to such errors.

Few approaches initially tried to solve this “communication and knowledge gap” as framed by product and engineering teams were to design collaterals (user manuals, one pagers, self troubleshooting guides) followed by a rule based chatbot.

Discovery

1. Learnings from previous approaches

Collaterals

- Had right information, but they were too complex to actually be followed and used by end users. The steps were written the way the team would explain something to each other, not the way a healthcare worker could understand and most sites did not have on site IT teams to ask help if they got stuck or faced any issue.
- Next challenge was accessibility - to make sure they are circulated with the users at the right time via WhatsApp groups and then making sure users have them hand when some issue came up.

[attachment:5a62869d-c20d-4130-a6c6-c9d3c5736841:Collateral_Example.pdf](#)

Collateral Examples

Rule based Chatbot

-
- ```

graph TD
 User((User)) --> QR[User scans the QR CODE]
 QR --> WhatsApp[WhatsApp Chat Opens]
 WhatsApp --> EnterPhone[Enter phone no.]
 EnterPhone --> CheckStatus[Previous conversations already exist]
 CheckStatus --> SendCode[Send verification code]
 SendCode --> EnterCode[Enter code]
 EnterCode --> VerifyCode[Verify code]
 VerifyCode --> AddToChat[Add to WhatsApp chat]
 AddToChat --> SendMsg[Send message]
 SendMsg --> AgentResp[Agent responds]
 AgentResp --> SendProblem[Send problem]
 SendProblem --> AgentSol[Agent provides solution]
 AgentSol --> RateService[Rate service]
 RateService --> ThankYou[Thank you for rating]

```
- The flowchart illustrates a user journey for a mobile application. It begins with a user scanning a QR code, which triggers the opening of a WhatsApp chat. The user then enters their phone number, and the system checks if previous conversations already exist. If not, a verification code is sent, which the user enters. The system then verifies the code and adds the user to a WhatsApp chat. The user sends a message, and the agent responds. The user then sends a message about a problem, and the agent provides a solution. The user is then asked to rate the service, and the user provides a rating. The journey ends with the user being redirected to a 'Thank you for rating' screen.



The analysis shared by product and engineering team did not show the full picture. They often shared data from Jira by exporting the relevant fields. All the data was technical and quantitative there was no clear why for any type of issues. So, I started going through the Jira tickets myself one by one to simply understand both client and internal team side perspective.

support experience for onprem systems with no reliable internet

They fill the other jira field where they are supposed to categorise the issue level (L1, L2, L3) and issue type not an issue, client side issue, issue. It further had categories like knowledge gap, communication gap, expected behaviour of product. These fields were little inconsistent throughout the tickets and was subjective to interpretation specially in L1 type issues.

### 3. Working with Techops

I collaborated with techops to understand what was the cause of these issues in simpler language. The purpose of this exercise was to break down technical issue in simpler language. I categorised these issues into symptom versus cause framework. What the user reported - what did they experience, versus what was actually going on underneath. Asking "why" for every issue showed different levels of each issue. What could really be understood by a non technical person and where will they need help. This gave a lot of clarity which was not possible to get by just looking at the jira tickets analysis.

**Cause & Symptom Framework**

| Issue                                       | Symptom (User Perspective)                        | Cause                                                                                                               |
|---------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Docker not running in background            | qTrack app not opening                            | Background app fails to start or load                                                                               |
| Unable to log in to qTrack                  | Users not able to log in                          | Browser cache needs to be cleared                                                                                   |
| Unable to log in to qTrack                  | Users not able to log in                          | Wrong password entered. Passwords are long and complex with special characters, leading to human error while typing |
| Blank/white screen on opening qTrack        | qTrack app not opening or loading, appears broken | Previously sent scans take time to load. Can take 5 to 7 minutes depending on scan volume already in the app        |
| Network switching between saved connections | Scans not visible on qTrack, uploads failing      | Device silently switches to a different saved wifi network, breaking the connection between systems                 |
| Scans not visible on qTrack (general)       | No scans appearing after being taken              | Could be network disconnect, but also other causes. No single root cause                                            |
| Scans stuck in processing                   | Upload shows as stuck, no progress shown          | Bulk upload with no visible queue or progress indicator                                                             |
| AI processing slow                          | Results taking a long time to appear              | Processing load increases as scan volume in queue increases                                                         |
| Running out of storage                      | App slows down or fails unexpectedly              | Local device storage filling up over time, not visible to the user                                                  |
| qTrack loading time on first open           | "Something is wrong, it's not working"            | 5 minute wait. Normal load time after power-on, no loading indicator shown                                          |

How issues were added in jira were little open to interpretation the person who is assigned the ticket and resolving that issue.

- **Mislabeled** - The product simply wasn't telling users what was happening, so a normal delay in AI processing or background app not starting looked like a

failure. Example - LAN cable unplugged from the laptop

- **What users could during trainings** - things users could genuinely learn, like not uninstalling a background app the system depended on, how to export CSV (feature in qTrack). Issues related to how to navigate product were also raised but that was rare.
- **Out of scope of frontline healthcare workers** - issues like understanding wifi switching between saved networks, which required some understanding infrastructure knowledge that had nothing to do with a healthcare worker's actual job.

---

## Core Insight

**The issues raised by end users were not happening due to communication / knowledge gap or poor understanding of the product. For users, they were not getting any next step information if something went wrong. The qTrack was not communicating if something went wrong or if any error was about to occur.**

The end users of qTrack are front line healthcare workers in their day to day they are not expected to deal with such issues or resolve them. That is not what they do. When something did not work the product stopped working or should some technical message which is not something end users can understand. These screening camps are high stress environments the product not working any way means screening would have to be stopped so in such cases end users acct from panic and immediately report the issue, which is a natural behaviour.

And even where training could technically close the gap, it was not a great experience for health care workers to start learning about engineering errors.

---

## Scoping the opportunity

- **Issues categorised** based on:
  - How common that issue is
  - How is it impacting users

The issues prioritized were technical in nature, but the reason they were prioritized was how often they occurred and how negatively they impacted users, whether they added friction to getting into qTrack, or stopped the on-ground workflow midway. In both cases, the product wasn't signaling anything to the user: not that something had gone wrong, not what to do about it, not even a way to tell if something was wrong at all.



- **Issues picked**

Issues shortlisted to start the design work based on what can be controlled by Qure and effort required (engineering + design).

## Technical Challenges

1. No single issue on this list had one clean root cause. "qTrack not opening" could mean five different things underneath, so there was no single fix to jump straight to. Instead, users needed a way to work through possibilities one at a time - ruling things out until the actual cause was clear, with the app guiding them toward the next step at each point.early and telling the user what was happening, instead of solving the switch itself.
2. The network disconnect or switching between saved network was the starting cause for so many issues. Engineering shared this cannot be automated - detected and fixed directly, without user getting involved. This is where things get complicated its difficult for non technical health are workers understand networking. And it was not even worth it because teaching this to end users was requiring more time than actually getting familiar the product qTrack. Not a great experience if users spend more time in learning about what type of issues can occur than engaging with the product.

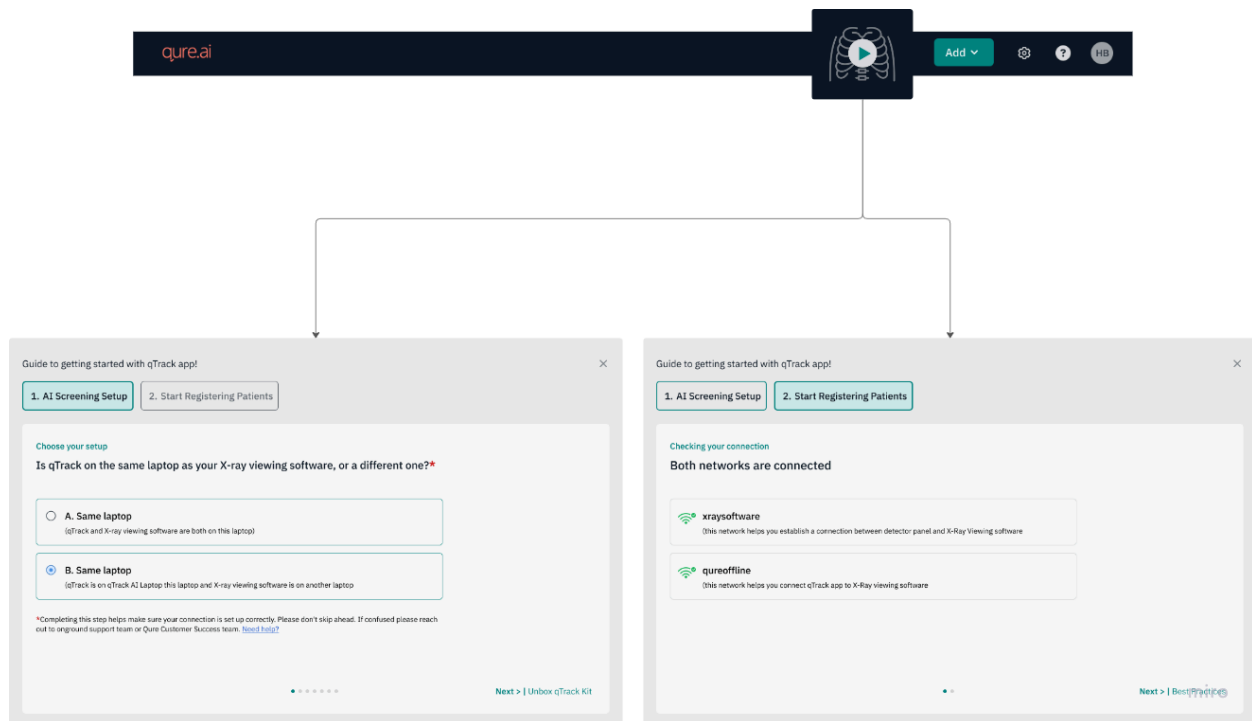
---

## Design solution

With so many uncertainties there was a need of an environment which supports users in their day to day tasks and give them preventive measures to reduce the chances of errors. That is how the 'Start Screening' button was added to mimic the workflow of onground users and guide them. It is divided into 2 main sections

-

1. **AI Screening Setup**
2. **Start Registering Patients**



Note: This case study covers “start registering patients” in more detail. To understand how section one was created [please read here](#)

## How workflow looks for the onground users

### Users start with the AI screening Setup

While connecting their equipment during onboarding, the user names the network they're on, directly in the app. It's a small addition to a step they're already doing, not a separate task. That name gets saved, and becomes the reference point for everything that follows.

*(Note: This onboarding flow, AI Screening Setup, is covered in full detail in the other case study. The network name field is the one addition made here, for this project.)*

#### 1. Network Name Field

*(Addresses: Network disconnect between different machines/devices)*



An addition to the first section, AI Screening Setup (onboarding flow) an input field where users can add the network name for easy recall instead of getting into technicalities. In the setup, users name the networks they are connecting to, directly in the app. Ideally this stays the same every time, but these are mobile camps, so the team packs up and sets up again regularly, which naturally leads to small errors. Devices (laptops, mobiles, tablets) can also hold onto multiple saved networks, and if one connects automatically to a different saved network instead of the intended one, the connection between the two software systems breaks and has to be reconnected by reconfiguring the IP address. That's not something a non-technical user should be expected to do, no matter how clearly the steps are written. This couldn't be automated from the backend either, since multiple networks can be involved, with no way for the system to know which one is meant for what.

### **How it works**

Users name the network once, at setup. That name becomes the reference point the setup connection check compares against before a session starts, confirming the user is still on the network they originally intended before they move on to registering patients. Users can edit it any time something changes.

### **Users are ready to start registering patients**

#### **2. Network Connection Check & Best practices reminder**

*(Addresses: Network disconnect between different machines/devices)*

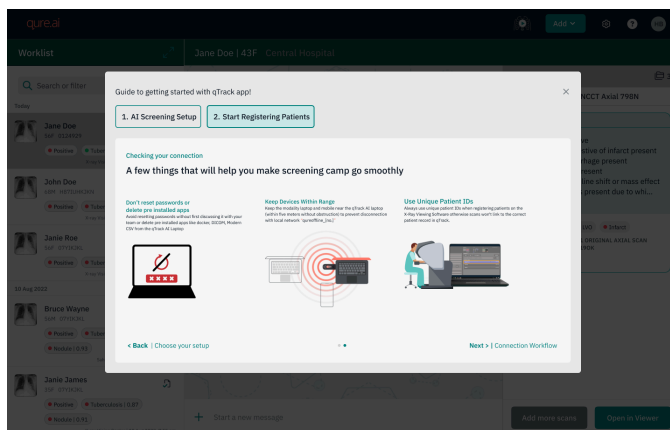
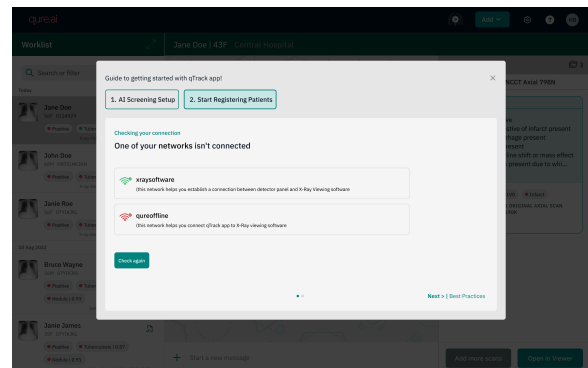
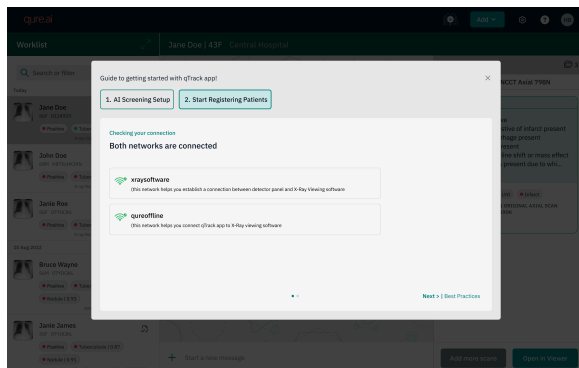
The next section in the Start Screening flow is "Start Registering Patients." This runs a few last minute checks before the user begins, and surfaces best practices for a smoother session.

Two states:

- Both connected - user sees both saved network names confirmed, and can move on.
- Mismatch in connection - the app names exactly which network is missing, rather than a generic "not connected" message, so the user knows what to check. A retry button lets them recheck without restarting the whole setup.

## How it works

Once the user has named their networks, the app checks their current connection status against those names once, at setup, before they start registering patients. This catches a mismatch upfront, before the TB screening camp begins, rather than letting it surface later as a "scans not visible" issue mid-session.



## Camp Screening session has started

### 3. Running Session Bar

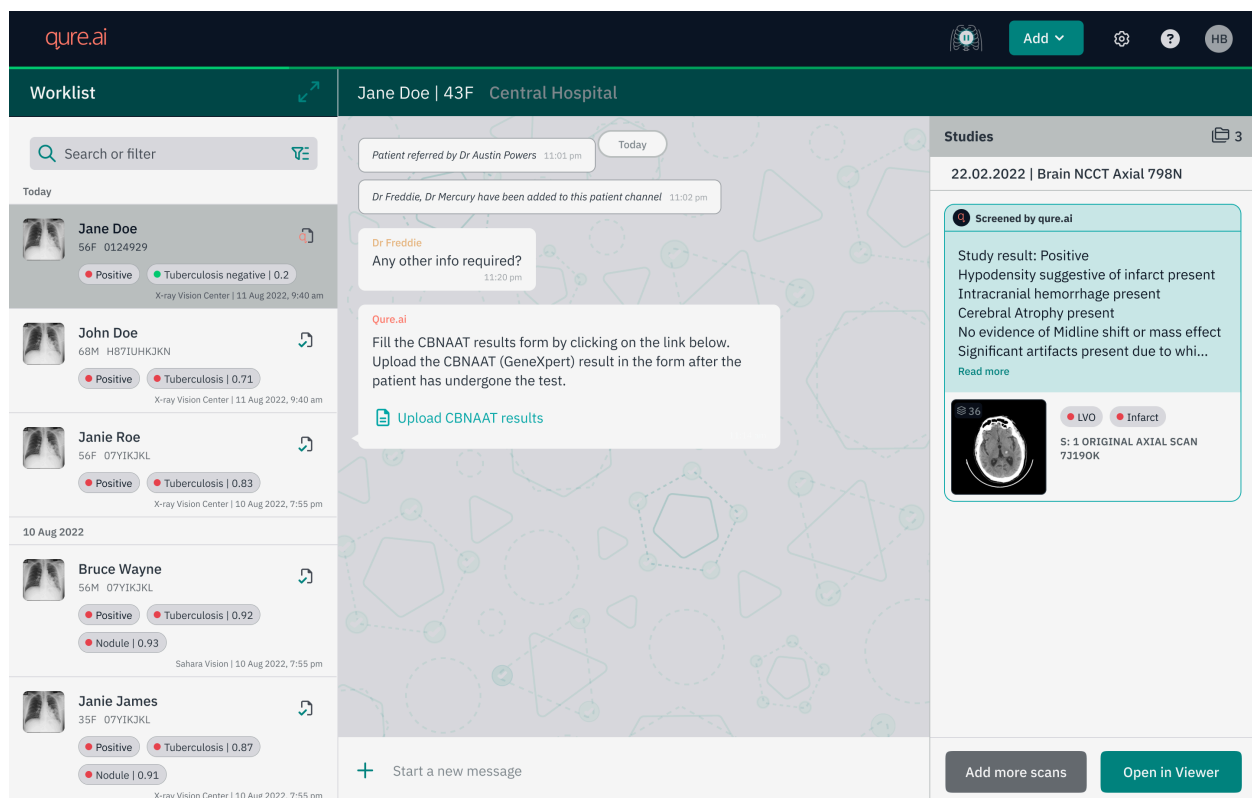
*(Enables: background monitoring during a session )*

A green bar runs across the top of the screen, just below the nav bar, for as long as a screening session is active. It does two things:

- **Signals that the session is live.** Since screening at a camp runs for hours at a stretch, the bar gives users a constant, ambient confirmation that they are still "in session" without needing to check anywhere else.
- **Triggers the background checks that run during a session** — like the session check-in prompt and network monitoring — since those only make sense while screening is actually active, not when a user is just browsing patient records.

## How it works

The bar appears as soon as a user taps "Start Screening" and completes setup. It stays on screen continuously until the user explicitly stops the session at the end of the day.



## Checking in between the session

#### 4. Session check in prompt

(Addresses: Scans not visible on qTrack or any any on ground issues)

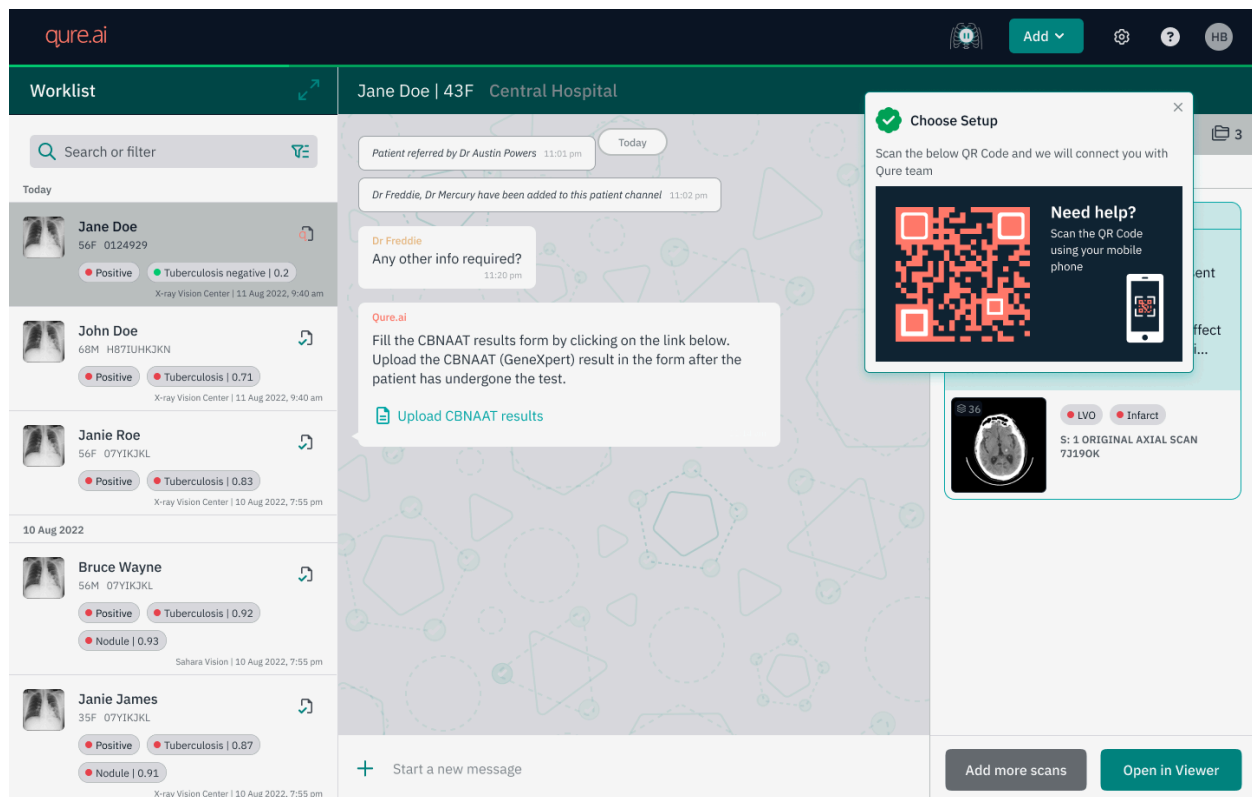
Scans not appearing on qTrack once pushed from the X-Ray Viewing software is a common issue and can happen for various reasons. With this check, we check -

- X-Ray scans not coming to qTrack - what's the reason? Did something break, or did something happen on the ground that needs help? This check verifies that everything on the ground is running smoothly.
- This also helps the Qure team to understand user activity by going through the logs.

#### How it works

- Once a user has started the screening session, the backend runs a check in the background for when the last scan was received on qTrack. If no scan comes in for 30 minutes straight, this check-in prompt appears for the user.

The screenshot displays the qure.ai web application interface. On the left, a 'Worklist' sidebar shows a list of patients with their names, IDs, and scan status. The main area shows the details for 'Jane Doe | 43F | Central Hospital'. A notification box titled 'Screening still on?' is visible, indicating that qTrack has not received a scan in the last 30 minutes. Below the notification, a 'Study result' section shows a positive result for tuberculosis and a negative result for intracranial hemorrhage. A 'Study result' section also shows a positive result for intracranial hemorrhage and a negative result for cerebral atrophy. A 'Study result' section also shows a positive result for intracranial hemorrhage and a negative result for cerebral atrophy. A 'Study result' section also shows a positive result for intracranial hemorrhage and a negative result for cerebral atrophy.



## 5. Mid Session Network change / switch

*(Addresses: Network disconnect between different machines/devices)*

If the network changes while a session is already active, qTrack names the change directly and surfaces it immediately so users can take action.

### How it works

Once a session is active, the app continues watching the network against the name saved at setup. If it detects a change, it alerts the user right away.